



OpenSmalltalk VM vs TruffleSqueak VM

Final Presentation

Recap: Outlook



- **OSVM**
 - Further advance **proxy inspector system**
 - Fix bugs for **Sista/Scorch MVP**
- **TSVM**
 - Finish and validate **Diff Tool for Truffle Compilation Traces**
 - Replace **Byte Array** representation with **Truffle String** too

Come into exchange with
us at the **WORLD CAFÉ**
☕
at 3.30pm



Master Project 2025

Marius Dörbandt, [Oliver Heß](#), [Antony Kamp](#), [Christoph Thiede](#)

Advisors: Marcel Taeumel, Jens Lincke, Robert Hirschfeld

OpenSmalltalk VM vs TruffleSqueak



After all the presentations:
Which car would you like to work on?

Last presentation	This presentation
Through the eyes of a VM Developer	Through the eyes of a VM Tools Developer



Recap: Motivation – Are We Fast Yet?



Input

iter := 300

size := 100

json := “{"head":{"requestCounter":4},...}”

Processing

for i in iter:

 timer.start()

 for j in size:

 parser := new Parser()

 parsed_json := parser.parse(json)

 verify(json, parsed_json)

 end

 timer.stop()

end

Pseudocode: AWFY JSON Benchmark

Cross-Language Compiler Benchmarking

Are We Fast Yet?

Stefan Marr Benoit Daloz Hanspeter Mössenböck

Johannes Kepler University Linz, Austria

{stefan.marr, benoit.daloz, hanspeter.moessenboeck}@jku.at

Abstract

Comparing the performance of programming languages is difficult because they differ in many aspects including preferred programming abstractions, available frameworks, and their runtime systems. Nonetheless, the question about relative performance comes up repeatedly in the research community, industry, and wider audience of enthusiasts.

This paper presents 14 benchmarks and a novel methodology to assess the compiler effectiveness across language implementations. Using a set of common language abstractions, the benchmarks are implemented in Java, JavaScript, Ruby, Crystal, Newspeak, and Smalltalk. We show that the benchmarks exhibit a wide range of characteristics using language-agnostic metrics. Using four different languages on top of the same compiler, we show that the benchmarks perform similarly and therefore allow for a comparison of compiler effectiveness across languages. Based on anecdotes, we argue that these benchmarks help language implementers to identify performance bugs and optimization potential by comparing to other language implementations.

Categories and Subject Descriptors D.3.4 [Programming Languages]: Processors—Compilers

With the wide variety of languages, there is no trivial answer. It is also unclear whether any single answer is useful outside of a specific context. Nonetheless, the question is asked again and again, which leads us to search for a way to answer one specific question: *is one implementation of a widely used set of core language abstractions as fast as other implementations of the same set of abstractions?*

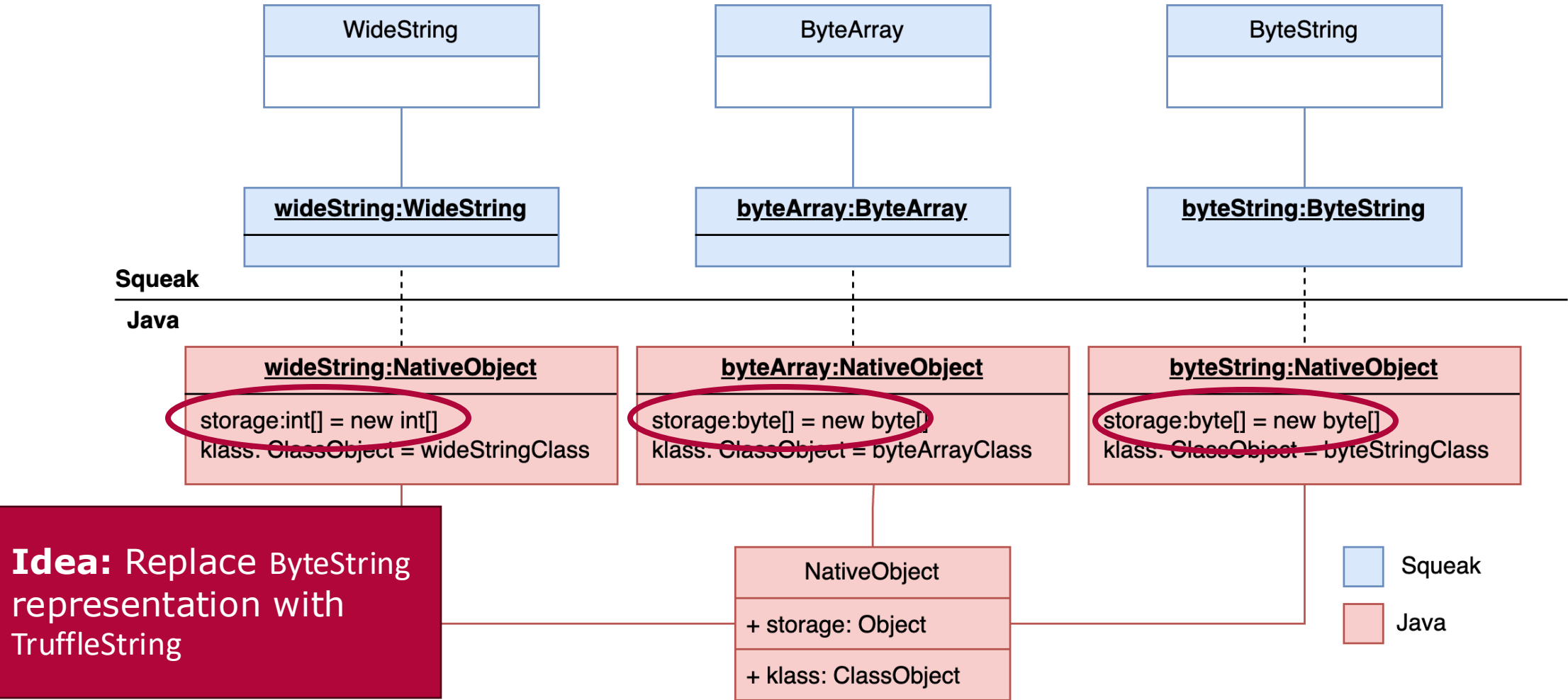
We conjecture that a commonly supported set of core language abstractions (incl. objects, polymorphic methods, closures, and arrays) is relevant for general application performance. Hence, it is necessary to optimize these abstractions to reach optimal performance. However, the set of abstractions is not universal. For example, programs in Haskell use language abstractions that are different from those in imperative languages. Another aspect is that optimizing only a set of core abstractions is not sufficient for optimal performance of all uses of a language. Programs typically use very expressive but language-specific abstractions, which need to be optimized as well. Nonetheless, a common set of abstractions likely minimizes the performance impact of different language semantics and enables a comparison.

This paper assesses whether we can use benchmarks written using only these core language abstractions for

TSVM: Data Representation

How are Squeak objects represented in Java?

Non-Primitive instances: Native Objects



TSVM: Why TruffleString?



-  Use given Truffle framework utilities
-  Improve interop behavior with language agnostic strings?
-  Improve runtime performance of string operations?

Introduce TruffleString #2505

New issue

✓ Closed



fniephaus opened on May 26, 2020 · edited by fniephaus

Edits Member ...

The goal of this issue is to publicly collect requirements and document the design and implementation of `TruffleString`, a language-agnostic string representation for string-like objects within Truffle languages.

Quoting @chumer:

The current design is immutable. Buffers are a different story we won't cover in the first design. [...] lots of Ruby requirements coming in from the Ruby team. Now `TruffleString` (is) in the hand of the regex team, which is naturally a cross language team.

Language Implementation	Type	Comments
Espresso	immutable	Implementation not (yet) public.
FastR	mutable	Uses <code>byte[]</code> and <code>String</code> within its <code>CharSXPWrapper</code> , which is used in <code>RStringVecNativeData</code>
GraalPython	immutable	Uses Java <code>CharSequence</code> as part of its <code>PString</code> .
Graal.js	immutable	Uses Java <code>CharSequence</code> within a <code>DynamicObject</code> as part of its <code>JSString</code> .

Assignees

 djoosoo

Labels

tracking truffle

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

TSVM: ByteString as TruffleString

v1

HPI

JSON Benchmark

	Original	v1
Average	201.466 ms	327.14 ms
Min	173 ms	291 ms
Max	1 532 ms	2 883 ms
Total	60 440 ms	98 142 ms

Table: JSON Benchmark Comparison

 
1.6x!!! slower

Follow-up Questions:

1. How do we get an overall view on the diff of the performance/statistics?
2. Which methods/compilations changed the most?
3. How do I narrow down and focus on a specific method?

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)



Background: Truffle Compilation Trace Logs

Compiler option: `--engine.TraceCompilation`

AWFYVector>>#append:	Tier 1 Time	53(	33+19	)ms AST	79 Inlined	0Y	0N IR	637/	1574 CodeSize	10366 Addr	02
Behavior>>#new <split-635>	Tier 1 Time	15(	12+4	)ms AST	33 Inlined	0Y	1N IR	195/	377 CodeSize	1503 Addr	02
AWFYHashTable class>>#new	Tier 1 Time	9(	5+4	)ms AST	32 Inlined	0Y	2N IR	222/	387 CodeSize	1412 Addr	02
AWFYJsonObject>>#initialize	Tier 1 Time	16(	11+5	)ms AST	66 Inlined	0Y	3N IR	322/	556 CodeSize	2601 Addr	02
Behavior>>#new <split-608>	Tier 1 Time	19(	12+7	)ms AST	33 Inlined	0Y	1N IR	195/	377 CodeSize	1503 Addr	02
String>>#compareTo:	Tier 2 Time	35(	10+25	)ms AST	36 Inlined	0Y	0N IR	218/	242 CodeSize	897 Addr	02
AWFYJsonObject class>>#new	Tier 1 Time	16(	12+4	)ms AST	32 Inlined	0Y	2N IR	222/	387 CodeSize	1412 Addr	02
String>>#=	Tier 2 Time	65(	32+33	)ms AST	97 Inlined	2Y	0N IR	471/	349 CodeSize	1092 Addr	02

Compilation tier

Node size of AST

Code size

Compilation time

Method inlining

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)



How do we get an overall view on the diff of the performance/statistics?

Input: Base and target log file

Goal: Analyze diffs in general statistics

Implementation: Regex-based approach

Output: summary diffs CSV file

metric	value_in_base_file	value_in_target_file	diff
total_compilation_count	189	188	-1
AST_size_sum	90987	74061	-16926
compilation_time_sum (tier 1)	6948955	8333508	1384553
compilation_time_min (tier 1)	6622	6430	-192
compilation_time_average (tier 1)	58394.59	71840.59	13446.0
compilation_time_max (tier 1)	998987	1128766	129773
code_size_sum (tier 1)	304688	325563	20875
code_size_min (tier 1)	528	528	0
code_size_average (tier 1)	2604.17	2855.82	251.65000000000001
code_size_max (tier 1)	17693	17713	20
compilation_time_sum (tier 2)	10044058	21799170	11755112
compilation_time_min (tier 2)	4446	3287	-1159
compilation_time_average (tier 2)	143486.55	302766.25	159279.7
compilation_time_max (tier 2)	755990	1220499	464509
code_size_sum (tier 2)	742502	946422	203920
code_size_min (tier 2)	280	280	0
code_size_average (tier 2)	11250.03	14339.73	3089.6999999999999
code_size_max (tier 2)	112814	107907	-4907

```
AST node statistics
Truffle node count      : count= 183, sum= 74061
Truffle call count      : count= 183, sum= 1554
  Indirect               : count= 183, sum= 2
  Direct                 : count= 183, sum= 1552
    Dispatched           : count= 183, sum= 779
    Inlined               : count= 183, sum= 773
    -----
    Cloned                : count= 183, sum= 0
    Not Cloned            : count= 183, sum= 1553
Truffle loops           : count= 183, sum= 0
-----

Compilation Tier 1
Compilation Rate         : 39066.74 bytes/second
Truffle Tier Rate        : 77117.58 bytes/second
Gaal Tier Rate           : 177192.80 bytes/second
Installation Rate        : 1092090.49 bytes/second
Time for compilation (us) : count= 116, sum= 8333508
Truffle Tier (us)        : count= 114, sum= 4221644
Gaal Tier (us)           : count= 114, sum= 1837337
Code Installation (us)   : count= 114, sum= 298109
Gaal node count          :
  After Truffle Tier      : count= 116, sum= 38991
  After Gaal Tier         : count= 116, sum= 76136
Gaal compilation result  :
Code size                 : count= 114, sum= 325563
Total frame size          : count= 114, sum= 14016
Exception handlers        : count= 114, sum= 428
```


TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)

Which methods/compilations changed the most?

Input: Base and target log file + target metric

Goal: Analyze metric-specific diffs and create ranking

Implementation:

Available metrics
AST size, Code size,
Compilation time,
Inlining information

1. Regex-based approach
2. In each compilation tier, sequentially match compilation occurrences that are present in both traces
3. Calculate diffs per match and aggregate for output

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)

Which methods/compilations changed the most?

Output A: metric-specific diff CSV file

method_name	compilation_occurence	compilation_tier	code_size_diff
SequenceableCollection>>#copyReplaceFrom:to:with:	1	1	5773
String>>#compareTo:	1	2	5942
AWFYJsonParser>>#endCapture	1	2	6064
String>>#=	1	2	6464
AWFYJsonParser>>#readChar:	1	2	6721

Output B: metric-specific colored version of target file (HTML)

String>>#compareTo:	Tier 2	Time 184(24+160)ms	AST 41	Inlined 0Y 0N	IR 1217/ 1698	CodeSize 6839	Ac
String>>#=	Tier 2	Time 264(74+191)ms	AST 113	Inlined 2Y 0N	IR 2330/ 1870	CodeSize 7556	Ac
Character>>#asString	Tier 2	Time 94(57+36)ms	AST 26	Inlined 4Y 1N	IR 532/ 677	CodeSize 2426	Ac
SequenceableCollection>>#atAllPut:	Tier 2	Time 52(23+29)ms	AST 102	Inlined 0Y 0N	IR 239/ 363	CodeSize 1549	Ac
AWFYJsonParser>>#read	Tier 2	Time 362(147+215)ms	AST 159	Inlined 9Y 1N	IR 3278/ 1943	CodeSize 7514	Ac
AWFYJsonParser>>#readChar:	Tier 2	Time 660(205+455)ms	AST 49	Inlined 13Y 1N	IR 5691/ 2769	CodeSize 10393	Ac
AWFYJsonParser>>#readTrue	Tier 1	Time 57(45+12)ms	AST 102	Inlined 0Y 5N	IR 353/ 687	CodeSize 2570	Ac
AWFYJsonLiteral class>>#TRUE	Tier 1	Time 12(7+5)ms	AST 13	Inlined 0Y 0N	IR 129/ 160	CodeSize 587	Ac
AWFYJsonParser>>#readStringInternal	Tier 2	Time 1219(302+916)ms	AST 147	Inlined 27Y 14N	IR 9987/ 9546	CodeSize 42992	Ac
AWFYVector>>#append:	Tier 2	Time 76(43+34)ms	AST 79	Inlined 0Y 0N	IR 547/ 960	CodeSize 6411	Ac
AWFYJsonParser>>#skipWhiteSpace	Tier 2	Time 1087(133+953)ms	AST 23	Inlined 17Y 0N	IR 9776/ 3396	CodeSize 12272	Ac
AWFYJsonParser>>#readArrayElement:	Tier 2	Time 976(292+683)ms	AST 73	Inlined 24Y 57N	IR 10408/ 10745	CodeSize 47055	Ac
AWFYJsonParser>>#readValue	Tier 2	Time 1070(325+745)ms	AST 701	Inlined 23Y 44N	IR 10609/ 9863	CodeSize 39613	Ac
AWFYJsonArray>>#add:	Tier 2	Time 82(48+33)ms	AST 45	Inlined 1Y 0N	IR 558/ 1033	CodeSize 6439	Ac

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)

How do I narrow down and focus on a specific method?

Input: Base and target log file + method name

Goal: Filter traces for method and detect occurrence spikes

Implementation:

1. Regex-based approach
2. Filter in both traces for method matches
3. Aggregate number of compilations/deoptimization/invalidations for both traces
4. Calculate diffs regarding these aggregations

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)

How do I narrow down and focus on a specific method?

Output A: method-specific diff CSV file

String>>#compareWith:	value_in_base	value_in_target_file	diff
total occurrences	2	2	0
total compilations	2	2	0
total compilations (tier 1)	1	1	0
total compilations (tier 2)	1	1	0
total deopts	0	0	0
total invalidations	0	0	0

Output B: colored versions of both log files (HTML)

Behavior>>#new <split-605>	Tier 1	Time	14(9+6)ms	AST	33	Inlined	0Y	1N	IR	195/	377	CodeSize	1503	Add
AWFYVector>>#append:														
String class>>#new:	Tier 2	Time	119(27+92)ms	AST	65	Inlined	2Y	1N	IR	176/	530	CodeSize	2043	Add
String>>#compareWith:	Tier 2	Time	184(24+160)ms	AST	41	Inlined	0Y	0N	IR	1217/	1698	CodeSize	6839	Add
String>>#=	Tier 2	Time	264(74+191)ms	AST	113	Inlined	2Y	0N	IR	2330/	1870	CodeSize	7556	Add
Character>>#asString	Tier 2	Time	94(57+36)ms	AST	26	Inlined	4Y	1N	IR	532/	677	CodeSize	2426	Add
SequenceableCollection>>#atAllPut:	Tier 2	Time	52(23+29)ms	AST	102	Inlined	0Y	0N	IR	239/	363	CodeSize	1549	Add
AWFYJsonParser>>#read	Tier 2	Time	362(147+215)ms	AST	159	Inlined	9Y	1N	IR	3278/	1943	CodeSize	7514	Add

TSVM: Proposal LCTLCT (Legendary compilation trace log comparison tool)



Discussion & Future work

Strengths

- More visible approach than regular log files
- Flexible in usage and able to answer different analytical questions
- Produces CSV output suitable for further data science tasks

Challenges

- **Regex** needs to be updated if format changes

Future work

- **Improve diff mechanism** not only match occurrences present in both logs (analogy: full outer join in SQL)

Follow-up Questions:

1. How do we get an overall view on the diff of the performance/statistics?
 - Observation: Code size increased!
2. Which methods/compilations changed the most?
 - Observation: Code size of string comparison methods increased!
3. How do I narrow down and focus on a specific method?
 - Observation for string comparison: No change in number of compilation or deoptimization

→ Analyze compilation graph of string comparison method in IGV

TSVM: ByteString as TruffleString

v1

HPI

🔍 *Observation in IGV*

String>>#compareTo: code size increased

⚡ *Reason:*

Comparing different types requires
a specialized node for each permutation

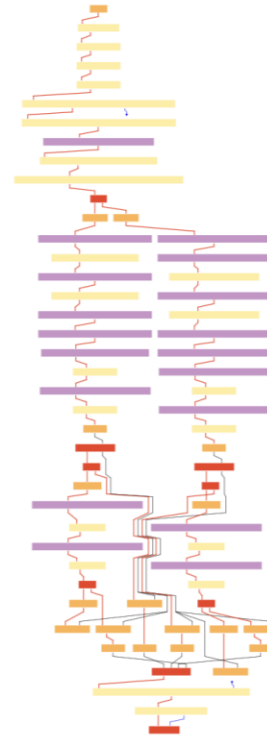


Image: Compilation Graph Original

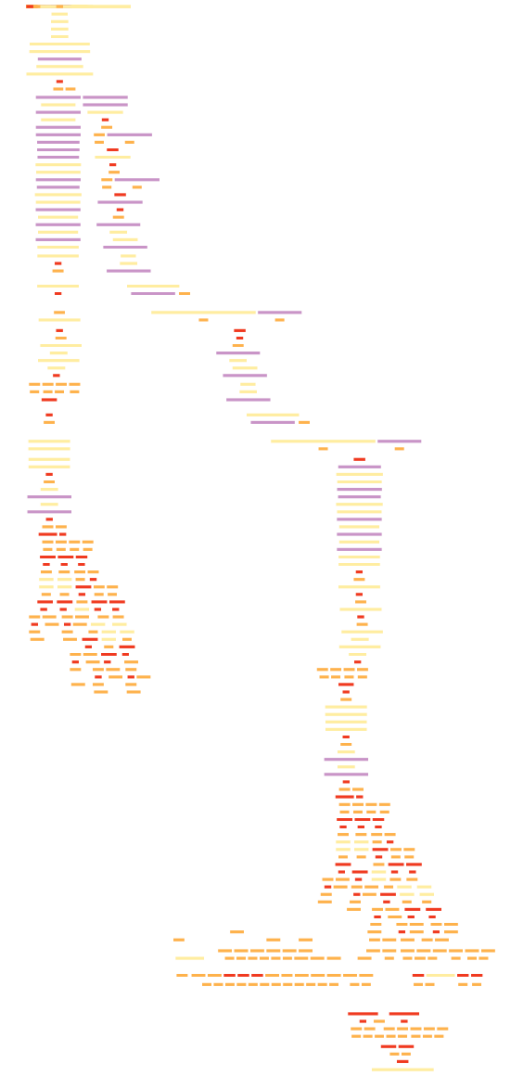


Image: Compilation Graph v1

TSVM: ByteArray as TruffleString

v2

HPI

JSON Benchmark

	Original	v1	v2
Average	201.466ms	327.14ms	292.2ms
Min	173 ms	291 ms	261 ms
Max	1 532 ms	2 883 ms	2 393 ms
Total	60 440 ms	98 142 ms	87 660 ms

Table: JSON Benchmark Comparison


**From v1 to v2
1.1x faster!!!**

Analysis

- Code size **decreased** in comparison to v1
- Code size **increased** in comparison to original
- Benchmark **improved** in comparison to v1
- Benchmark **slower** in comparison to original

TSVM: Workflow

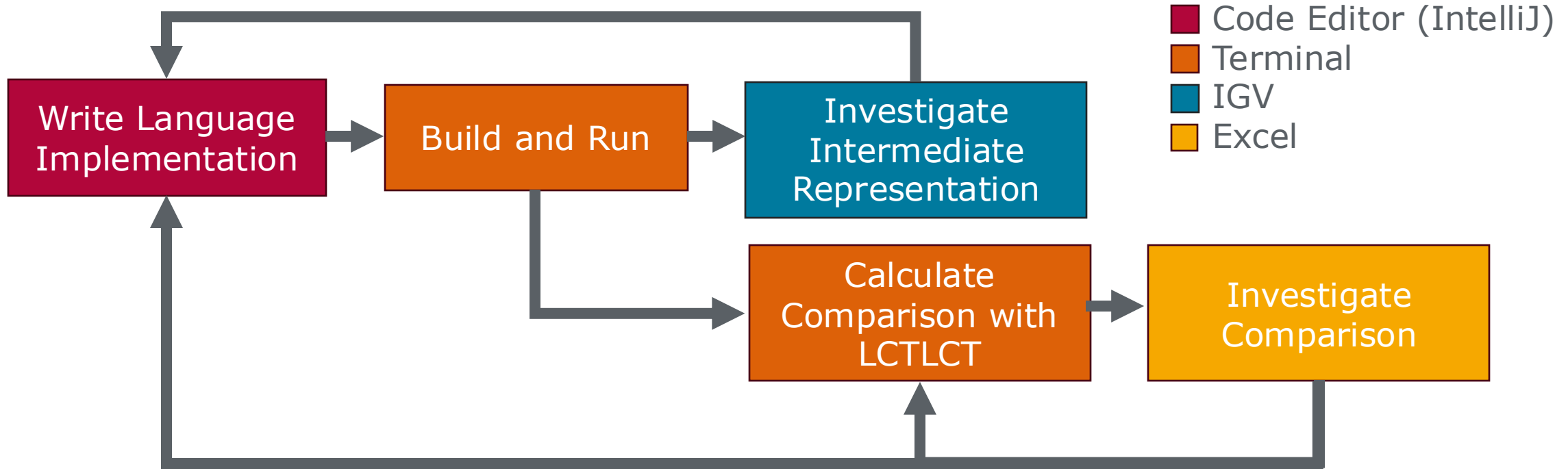


Image: Workflow developing VM

Observation

- Switching between IDE, terminal, IGV, LCTLCT, ... break development workflow
- 💡 Integrate all tools into our IDE
- 💡 Make switching as quick as possible

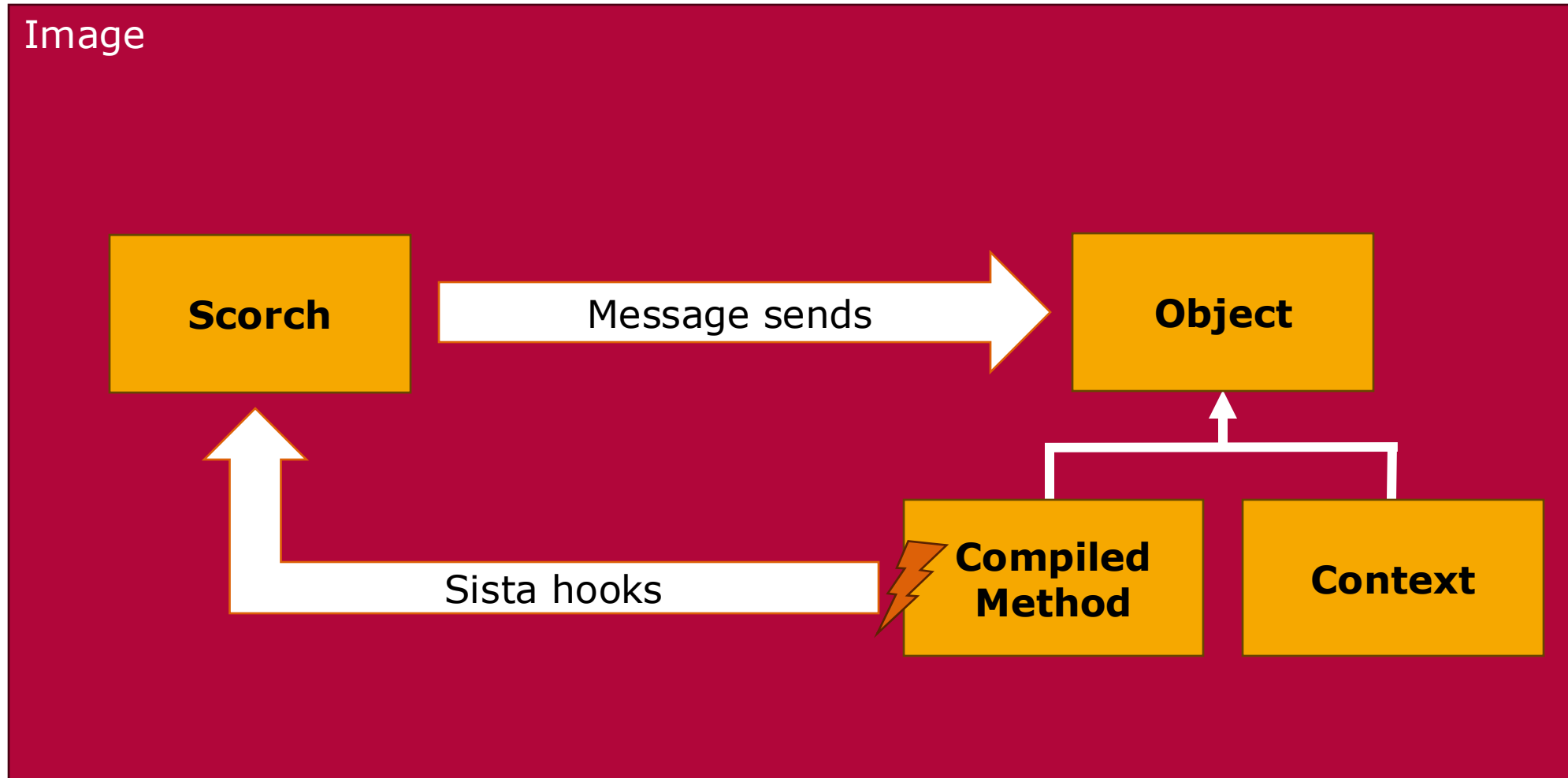


TSVM

- **Sista VM** (**S**peculative **I**ncline **S**mall**T**alk **A**rchitecture) [Bera, 2017a]
 - Optimizes frequent methods with **inlining** and **unsafe bytecodes**
 - Promises **speed-ups of 3x-5x**
 - **Adaptive optimizer** (Scorch) is implemented **in the image** to support live development and interactive debugging [Bera, 2017b]
- **Challenge: Bootstrapping** Sista/Scorch
 - **Frequent VM faults/crashes** impede initial development
- **Opportunity:** Use the **OSVM simulator** to debug the VM in another image [Miranda, 2018]
- **Questions:**
 - How can we run and develop Scorch in the **simulator**?
 - How can we **improve surrounding tooling** for exploratory programming in the simulator?

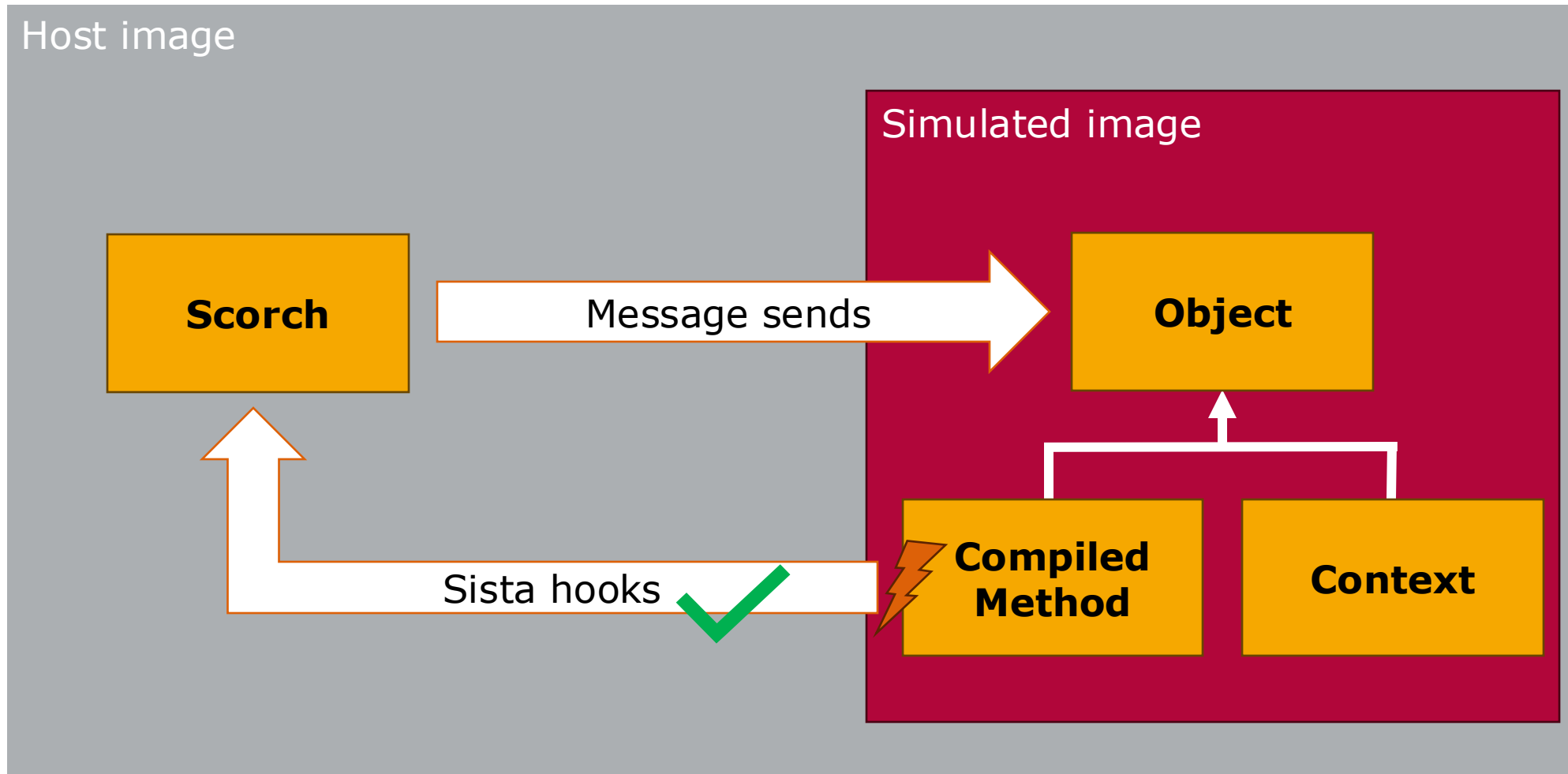
OSVM: Bootstrapping Scorch

Current Scorch/Image Interaction: Scorch Runs in Production Image



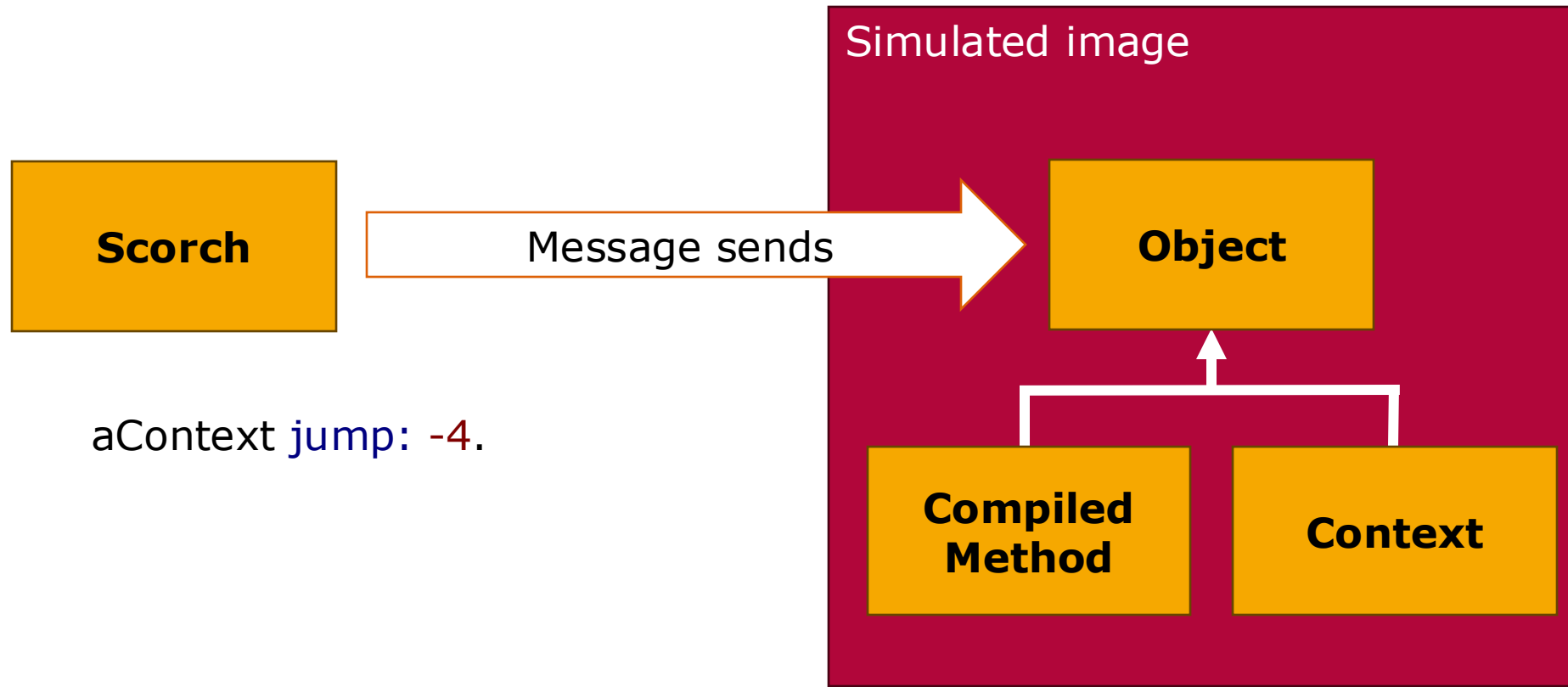
OSVM: Bootstrapping Scorch

Intended Scorch/Image Interaction: Develop Scorch Next to Simulated Image



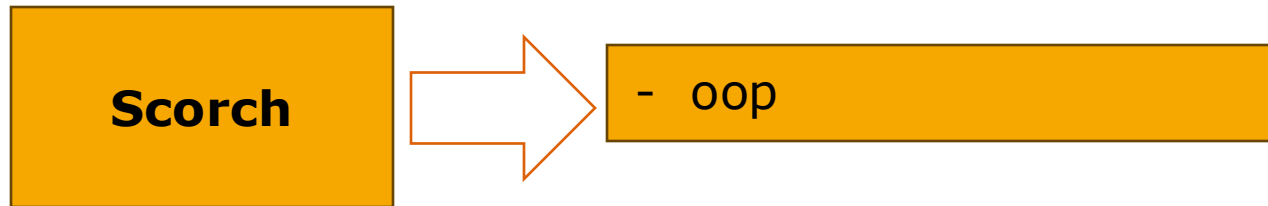
OSVM: Bootstrapping Scorch

Intended Scorch/Image Interaction: Develop Scorch Next to Simulated Image



OSVM: Bootstrapping Scorch

Intended Scorch/Image Interaction: Develop Scorch Next to Simulated Image

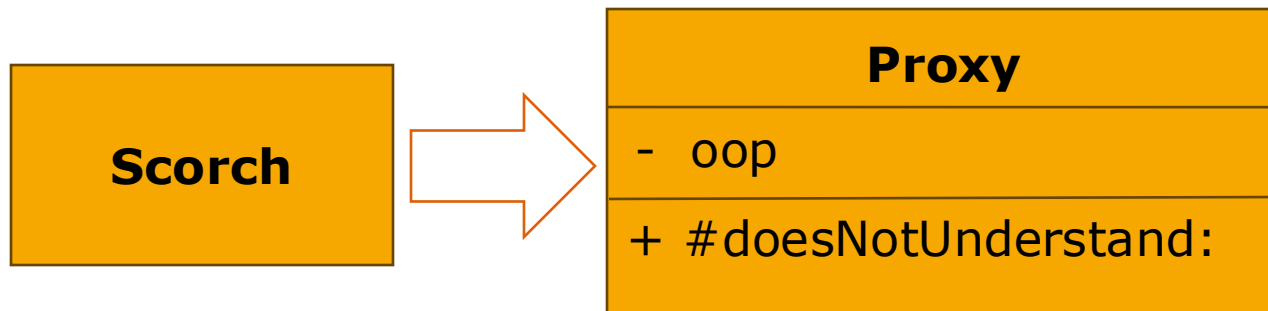


aContext **jump: -4.**

ObjectMemory
- bitmap
+ #fetchPointer:ofObject:
+ #storePointer:ofObject:withValue:

OSVM: Bootstrapping Scorch

Implementing a Transparent Proxy Framework

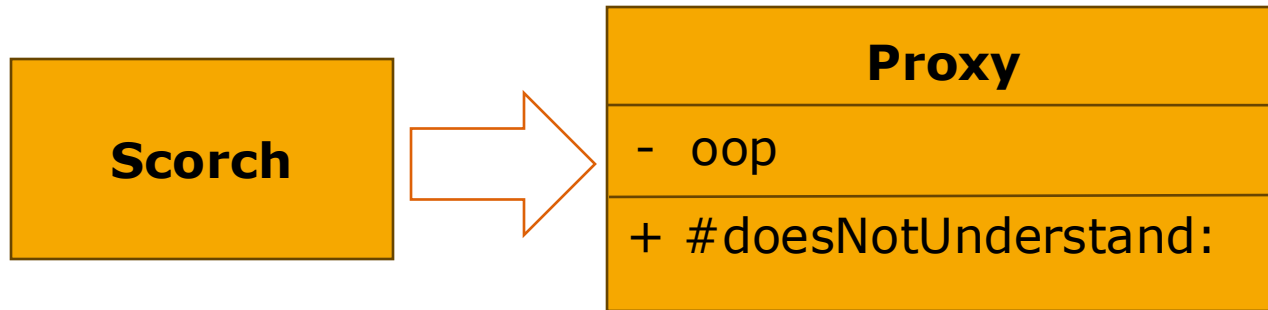


`aContextProxy jump: -4.`

ObjectMemory
- bitmap
+ #fetchPointer:ofObject:
+ #storePointer:ofObject:withValue:

OSVM: Bootstrapping Scorch

Implementing a Transparent Proxy Framework



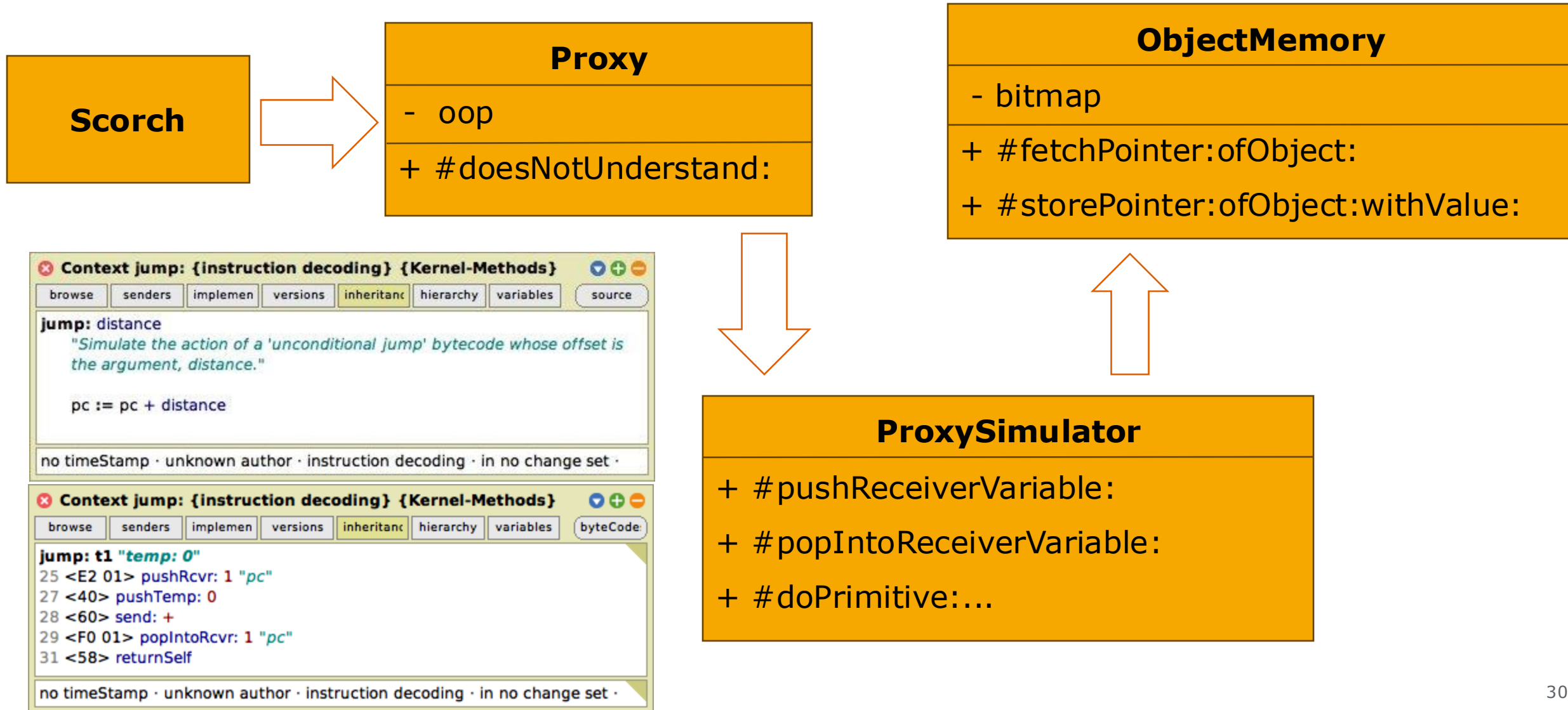
ObjectMemory
- bitmap
+ #fetchPointer:ofObject:
+ #storePointer:ofObject:withValue:

```
Context jump: {instruction decoding} {Kernel-Methods}
browse senders implemen versions inheritanc hierarchy variables source
jump: distance
  "Simulate the action of a 'unconditional jump' bytecode whose offset is
  the argument, distance."
  pc := pc + distance
no timeStamp · unknown author · instruction decoding · in no change set ·

Context jump: {instruction decoding} {Kernel-Methods}
browse senders implemen versions inheritanc hierarchy variables byteCode:
jump: t1 "temp: 0"
25 <E2 01> pushRcvr: 1 "pc"
27 <40> pushTemp: 0
28 <60> send: +
29 <F0 01> popIntoRcvr: 1 "pc"
31 <58> returnSelf
no timeStamp · unknown author · instruction decoding · in no change set ·
```

OSVM: Bootstrapping Scorch

Implementing a Transparent Proxy Framework



OSVM:

Bootstrapping Scorch

Running Benchmarks

Simulation of spurreader-64-4.1.image

Projects Tools Apps Do Extras Windows Help HomeProject Search or evaluate... 01:40:35

snapshot

Transcript

```
---QUIT---{21 July 2025 . 1:40:35 am} spurreader-64-4.1.image priorSource: 7550601
Smalltalk specialObjectsArray at: 59 put: nil

Transcript showIn: [10000 timesRepeat:
  [(JsonParser with: "Carpe Squeak") read; readStringInternal]]
  timeToRunWithoutGC

13
Smalltalk specialObjectsArray at: 59 put: #conditionalBranchCounterTrippedOn:

Transcript showIn: [10000 timesRepeat:
  [(JsonParser with: "Carpe Squeak") read; readStringInternal]]
  timeToRunWithoutGC

9
```

counter tripped in
16r2BEDB0 M ByteString class(String class)>new: 16rD6FED8: a(n) ByteString

counter tripped in
16r2BEE20 M ByteString(String)>= 16r755F48: a(n) ByteString

counter tripped in
16r2BEDE8 M String class>with: 16rD700B8: a(n) String

counter tripped in
16r2BEDA8 M ByteString class(String class)>new: 16rD6FED8: a(n) ByteString

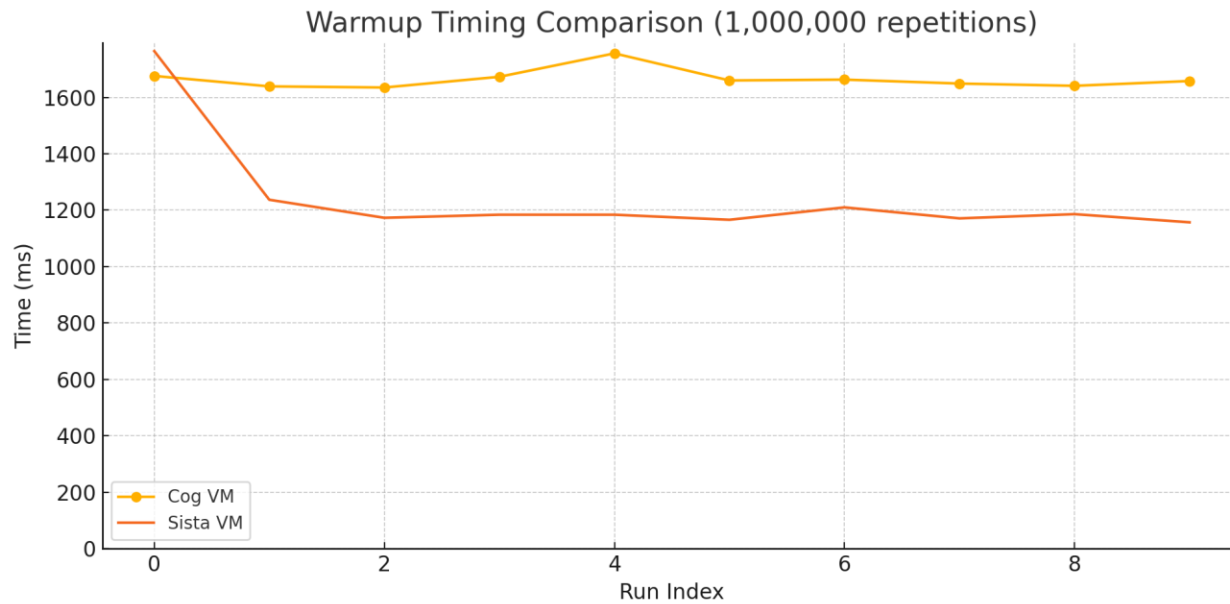
3,079,585/26,970

OSVM: Bootstrapping Scorch

Running Benchmarks



```
[1000000 timesRepeat:  
  [(JsonParser with: "'Carpe Squeak'") read; readStringInternal]]  
timeToRunWithoutGC
```



1.4x faster

	Cog	Sista
Avg	1 665 ms	1 243 ms
Min	1 635 ms	1 157 ms
Max	1 756 ms	1 764 ms
Total	16 650 ms	12 432 ms

Benchmark Specs
i5-6267U @ 2.9 GHz x 4, 6
GB RAM, Ubuntu 18.04

OSVM: Improving Exploratory Programming in the Simulator

Object Inspection



State of the art: **Low-level, non-interactive text output**

```
disassemble method/trampoline...
disassemble method/trampoline at pc
disassemble ext head frame method
print oop...
long print oop...
print context...
symbolic method...
inspect object memory
```

```
squeak> 16r872020: a(n) JsonObject
      16r11 =2 (16r2)    16r872038 an Array
16r872038: a(n) Array
      16rA0CE08 nil      16rA0CE08 nil      16r872260 an Association #level -> 16r00
      16rA0CE08 nil
```

Transcript output of simulator

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✕ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

(ReadStream>>#next "(vm)16r2043788: a
CompiledMethod")

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✕ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

"(vm)16rEA3348: class ReadStream"

next

<primitive: 65> "0"

position >= readLimit

ifTrue: [^ nil].

^ collection at: (position := position + 1)

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

-1152921504606781437

"primitive: 65

numArgs: 0

numTemps: 0

numLiterals: 3

frameSize: 16

bytecodeSet: SistaV1"

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✕ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

(vm)16rBED5F0: #ifTrue:ifFalse:

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

(vm)16r20A0200: an AdditionalMethodState

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

(vm)16rE86060: a ClassBinding #ReadStream ->
16r00EA3348

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

248

Evaluate expressions on inspected object

explore

Inspectors on method proxies

Reuse rich, domain-specific tools to inspect and modify objects of from simulated image

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

<primitive: 65>

33 <F8 41 00> callPrimitive: 65

36 <01> pushRcvr: 1 "position"

37 <02> pushRcvr: 2 "readLimit"

38 <65> send: >= [inline cache SmallInteger => SmallInteger>>#>=]

39 <C0> jumpFalse: 41 [branch taken 7% (4358/65535)]

40 <5B> return: nil

41 <00> pushRcvr: 0 "collection"

42 <01> pushRcvr: 1 "position"

43 <51> pushConstant: 1

44 <60> send: +

45 <F3 01> storeIntoRcvr: 1 "position"

47 <70> send: at: [polymorphic inline cache Array =>

Evaluate expressions on inspected object

explore

Advanced bytecode printer with **Sista** branch counters and syntax highlighting

OSVM: Improving Exploratory Programming in the Simulator



Object Inspection

✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

16r2043788 (is in old space)

Evaluate expressions on inspected object

explore

Additional VM-specific inspection fields

OSVM: Improving Exploratory Programming in the Simulator

Object Inspection



✖ (vm)16r2043788: a CompiledMethod

self

vm address

vm header

vm generated method

source code

decompiled

all bytecodes

header

literal1

literal2

literal3

33

34

35

16r71F32941C000468

0: 00000111 numSlots=16r7 (7)

8: 00 isMarked=false unused=true

10: 0111110011001010010100 identityHash=2044564

32: 000 isGrey=false isPinned=false isRemembered=false

35: 11100 format=28

40: 00 isImmutable=false unused=0

42: 0000000000010001101000 classIndex=16r468

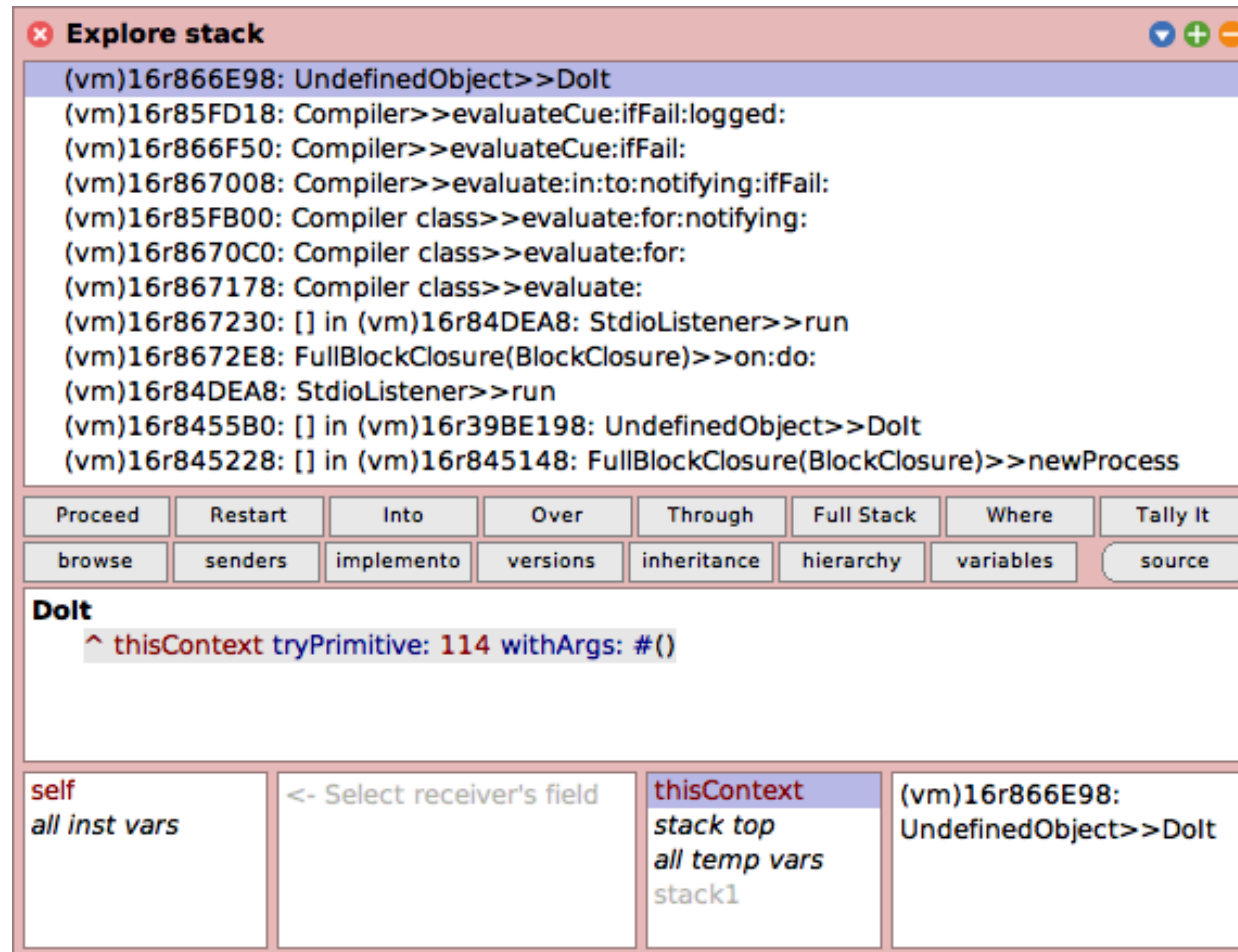
Evaluate expressions on inspected object

explore

Additional VM-specific inspection fields

OSVM: Improving Exploratory Programming in the Simulator

Debugging

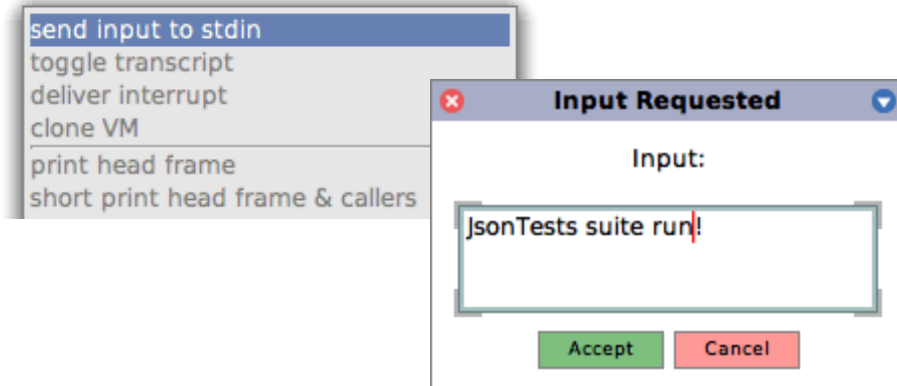


**Debuggers on
context proxies
(currently read-only)**

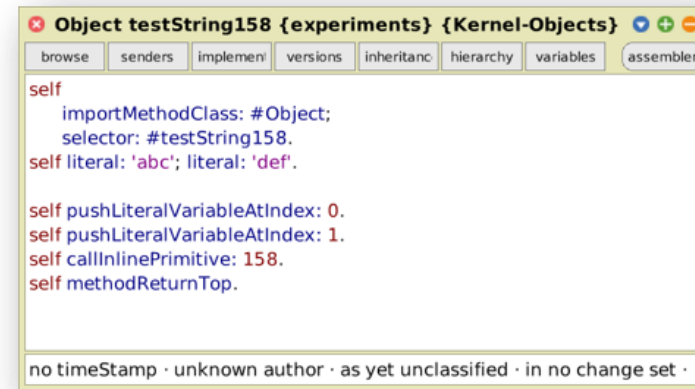
(Process forContext: aContextProxy)
debugWithTitle: 'Explore stack'

OSVM: Improving Exploratory Programming in the Simulator

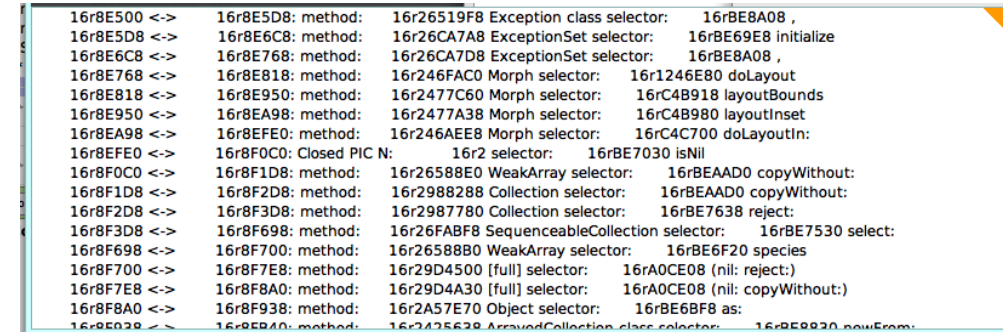
Other Contributions



Added non-modal REPL interface



New bytecode editor



Improved Transcript performance



Improved build chain & documentation

- **The road ahead for Sista/Scorch**
 - **Iteratively extend Sista/Scorch** with new unsafe bytecodes and optimizations using our proxy framework
 - **Harden and test** the new VM for a release
- **Open todos for our proxy framework**
 - Implement **missing edge cases** for context proxies
 - **Manipulation of top contexts** for external debugging
 - Complete trap support for **Scorch deoptimization**
 - **Accelerate** the proxy accesses by using **bytecode rewriting** instead of context simulation
- How could we further **improve program exploration** in the simulator?
 - Introduce **additional VM inspectors** for (unmarried) frames, stack pages, immediate values, ...?
 - Build an interactive, graphical **debugger for machine code?**

Comparison: Toolbuilding Experience in OSVM vs TSVM



How was our experience when extending tools?

	TSVM	OSVM
Tooling ecosystem	✗ Scattered ecosystem disrupts workflow continuity 💡 Integrate all tools in a single consistent, connected framework (e.g., an IntelliJ plugin)	✓ Overarching UI framework and runtime ✓ Openness by default: override methods, Inspector API

Comparison: Toolbuilding Experience in OSVM vs TSVM



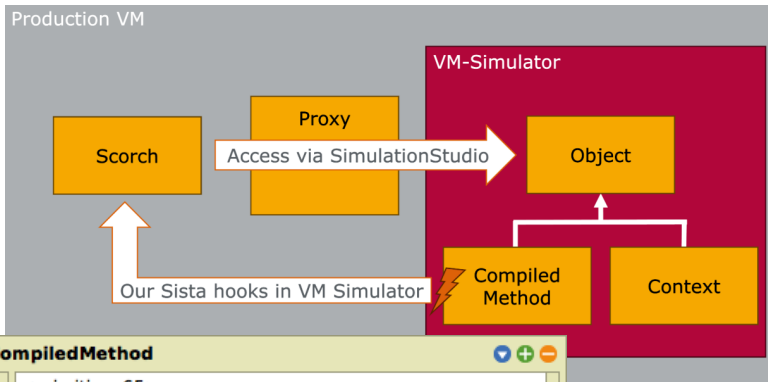
How was our experience when extending tools?

	TSVM	OSVM
Tooling ecosystem	✗ Scattered ecosystem disrupts workflow continuity 💡 Integrate all tools in a single consistent, connected framework (e.g., an IntelliJ plugin)	✓ Overarching UI framework and runtime ✓ Openness by default: override methods, Inspector API
Data Access	✓ Documentation of CLI ✓ Built-in, comprehensive logs ✗ Unstructured textual output 💡 Offer stable APIs (e.g., compilation traces as JSON)	✓ Openness by default: all objects are accessible ✗ Often no prepared interfaces ✗ Incomplete/scattered documentation ✗ Convoluted code impedes reverse engineering 💡 Emphasize domain-driven design 💡 Extend Slang support for higher-level language features

Conclusion



OSVM



```
(vm)16r2043788: a CompiledMethod
self
vm address
vm header
vm generated method
source code
decompiled
all bytetimes
header
<primitive: 65>
33 <F8 41 00> callPrimitive: 65
36 <01> pushRcvr: 1 "position"
37 <02> pushRcvr: 2 "readLimit"
38 <65> send: => [inline cache SmallInteger =>
SmallInteger>>#>=]
39 <C0> jumpFalse: 41 [branch taken 7% (4358/65535)]
40 <5B> return: nil
```

Simulation of spurreader-64-4.1.1.image

Projects Tools Apps Do Extras Windows Help

snapshot

	Cog	Sista
Avg	1 665 ms	1 243 ms
Min	1 635 ms	1 157 ms
Max	1 756 ms	1 764 ms
Total	16 650 ms	

Transcript

---QUIT---[21 July 2025 . 1:40:35 am]
Smailltalk specialObjectsArray at: 59 pu

Transcript shown: [10000 timesRepeat:
[(JsonParser with: "Carpe Squeak") read: readStringInternal]]
timeToRunWithoutGC

 **1.4x faster**

Discuss with us at the **WORLD CAFÉ** at 3.30pm 

TSVM



String>>#compareTo:	value_in_base	value_in_target_file	diff
total occurrences	2	2	0
total compilations	2	2	0
total compilations (tier 1)	1	1	0
total compilations (tier 2)	1	1	0

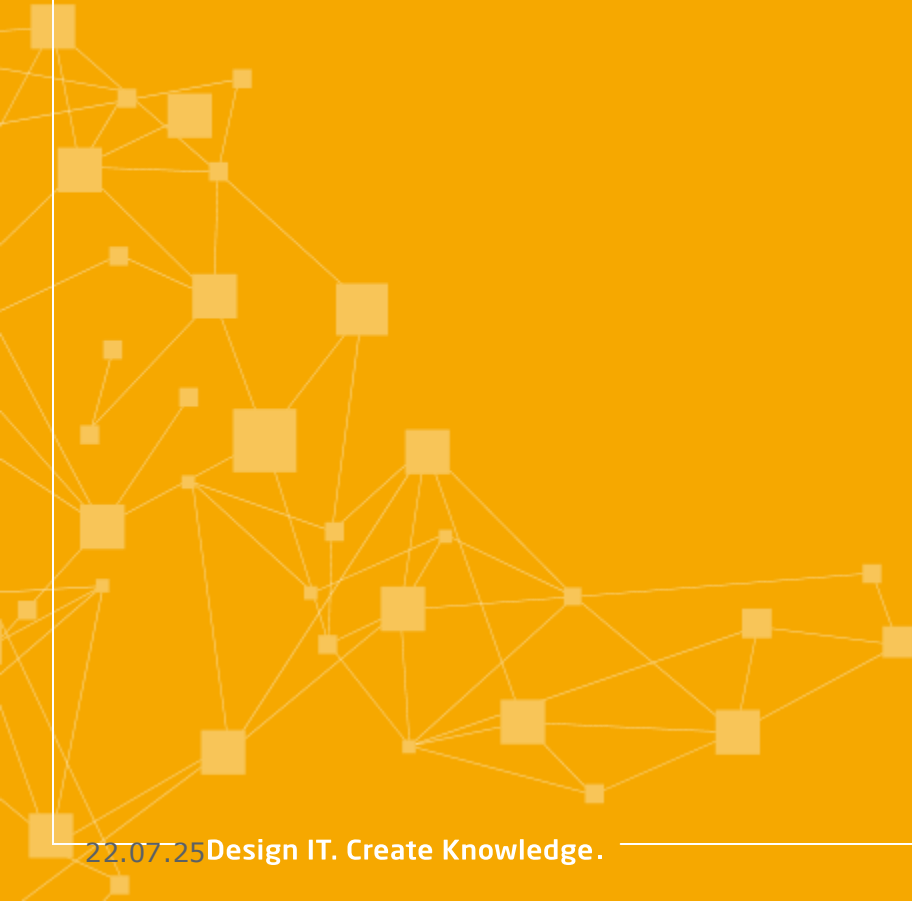
String>>#compareTo:	Tier 2	Time	184(24+160)ms	AST	41	Inlined	0Y	0N	IR	1217/ 1698	CodeSize	6839	Ac
String>>#=	Tier 2	Time	264(74+191)ms	AST	113	Inlined	2Y	0N	IR	2330/ 1870	CodeSize	7556	Ac
Character>>#asString	Tier 2	Time	94(57+36)ms	AST	26	Inlined	4Y	1N	IR	532/ 677	CodeSize	2426	Ac
SequenceableCollection>>#atAllPut:	Tier 2	Time	52(23+29)ms	AST	102	Inlined	0Y	0N	IR	239/ 363	CodeSize	1549	Ac
AWFYJsonParser>>#read	Tier 2	Time	362(147+215)ms	AST	159	Inlined	9Y	1N	IR	3278/ 1943	CodeSize	7514	Ac
AWFYJsonParser>>#readChar:	Tier 2	Time	660(205+455)ms	AST	49	Inlined	13Y	1N	IR	5691/ 2769	CodeSize	10393	Ac
AWFYJsonParser>>#readTrue	Tier 1	Time	57(45+12)ms	AST	102	Inlined	0Y	5N	IR	353/ 687	CodeSize	2570	Ac
AWFYJsonLiteral class>>#TRUE	Tier 1	Time	12(7+5)ms	AST	13	Inlined	0Y	0N	IR	129/ 160	CodeSize	587	Ac
AWFYJsonParser>>#readStringInternal	Tier 2	Time	1219(302+916)ms	AST	147	Inlined	27Y	14N	IR	9987/ 9546	CodeSize	42992	Ac
AWFYVector>>#append:	Tier 2	Time	76(43+34)ms	AST	79	Inlined	0Y	0N	IR	547/ 960	CodeSize	6411	Ac
AWFYJsonParser>>#skipWhiteSpace	Tier 2	Time	1087(133+953)ms	AST	23	Inlined	17Y	0N	IR	9776/ 3396	CodeSize	12272	Ac
AWFYJsonParser>>#readArrayElement:	Tier 2	Time	976(292+683)ms	AST	73	Inlined	24Y	57N	IR	10408/ 10745	CodeSize	47055	Ac
AWFYJsonParser>>#readValue	Tier 2	Time	1070(325+745)ms	AST	701	Inlined	23Y	44N	IR	10609/ 9863	CodeSize	39613	Ac
AWFYJsonArray>>#add:	Tier 2	Time	82(48+33)ms	AST	45	Inlined	1Y	0N	IR	558/ 1033	CodeSize	6439	Ac

TruffleString

	Original	v1	v2
Average	201.466 ms	327.14 ms	292.2 ms
Min	173 ms	291 ms	261 ms
Max	1 532 ms	2 883 ms	2 393 ms
Total	60 440 ms	98 142 ms	87 660 ms

- [Bera, 2017a] Clément Béra, Eliot Miranda, Tim Felgentreff, Marcus Denker, and Stéphane Ducasse. 2017. **Sista: Saving Optimized Code in Snapshots for Fast Start-Up**. In *Managed Languages and Runtimes (ManLang 2017)*. 11. <https://doi.org/10.1145/3132190.3132201>
- [Bera, 2017b] Clément Béra. 2017. **Sista: A Metacircular Architecture for Runtime Optimisation Persistence**. Programming Languages [cs.PL]. Dissertation, Université de Lille 1. <https://theses.hal.science/tel-01634137>
- [Miranda, 2018] Eliot Miranda, Clément Béra, Elisa Gonzalez Boix, and Dan Ingalls. 2018. **Two Decades of Smalltalk VM Development: Live VM Development through Simulation Tools**. In *Proceedings of the 10th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages (VMIL '18)*, November 4, 2018, Boston, MA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3281287.3281295>
- [Niephaus, 2022] Fabio Niephaus. 2022. **Exploratory Tool-building Platforms for Polyglot Virtual Machines**. Dissertation, Potsdam, Universität Potsdam. <https://publishup.uni-potsdam.de/frontdoor/index/index/docId/57177>
- [Würthinger, 2013] Thomas Würthinger et al. **One VM To Rule Them All**. In *Proceedings of the 2013 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software*. 2013. <https://dl.acm.org/doi/10.1145/2509578.2509581>

Appendix





OOPSIE: Object Oriented PointerS Interaction Engine



Inspectors on proxies

Syntax highlighting for jitted assembler methods

feat. Yaros-JavaScript bindings

Production VM

VM-Simulator

Scorch

Proxy

Access via SimulationStudio

Object

Compiled Method

Context

Our Sista hooks in VM Simulator



Bytecode editor: Write your own instructions!

Morphic across simulation boundaries

Non-modal REPL

Would you like to debug a proxy context?

Testing proxy methods in a debugger

Faster transcript

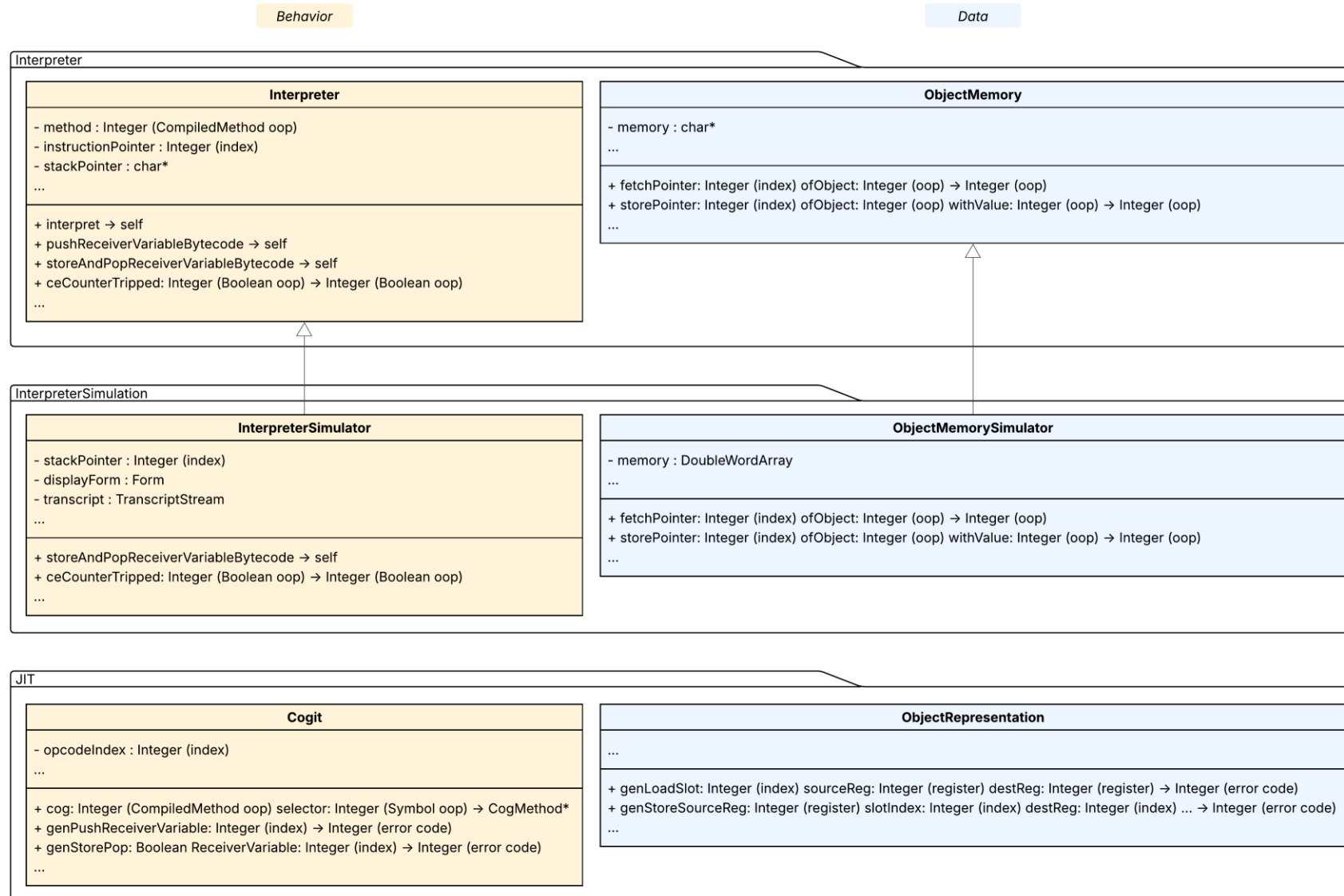
PROXIES ALL THE WAY DOWN

- **Scorch:** We ported the core of Scorch from Pharo to Squeak 6.1Alpha, made it compatible with our proxy simulator, and improved support for context simulation.
- **Squeak Trunk:**
 - **Transparent Proxy Support**
 - [resolveProxy.3.cs](#): Proposes a general pattern for resolving transparent proxies in primitive methods.
 - [Kernel-ct.1604](#): Fixes Boolean support in ObjectViewer (which currently functions as the de-facto reference implementation of transparent proxies in Squeak).
 - [Debugger step unexpected message sends.1.cs](#): Fixes stepping over ifTrue:/iffalse: sends to Boolean proxies in the debugger.
 - **Bytecode Representation and Execution**
 - [Kernel-ct.1599](#): Fixes and revises CompiledCode constructors for SistaV1 bytecode set.
 - [Kernel-ct.1600](#): Extends support for serializing CompiledCodes as storeStrings.
 - [Kernel-ct.1601](#): Implements and documents the rare bytecode pushActiveProcess.
 - [Kernel-ct.1605](#): Handles unusedBytecode trap from the VM by simulating unknown instructions in the context simulator.
 - [Kernel-ct.1606](#): Adds context simulation of primitiveExitToDebugger.
 - **Instruction Printing**
 - [InstructionPrinter with style.3.cs](#): Adds text styling to all instruction printers in the Trunk and VMMaker.

- Depends on:
 - [Collections-ct.1087](#): Adds Stream>>#isTextStream.
 - [addAttributesBack.7.cs](#): Adds reverse-ordered attribute accessors on Text and optimizes streaming of formatted texts to a lower complexity class.
- **VMMaker**
 - [VMMaker.oscog-mad.3558](#), [VMMaker.oscog-mad.3559](#): Fixes for sendAndBranchData (primitiveSistaMethodPICAndCounterData).
 - [VMMaker.oscog-ct.3562](#): Adds breakpoint in simulator when encountering an unknown instruction.
 - [VMMaker.oscog-ct.3541](#): Fixes and improves UI layout of VMMakerTool.
 - [VMMaker.oscog-ct.3556](#), [VMMaker.oscog-mad.3557](#), [VMMaker.oscog-ct.3563](#), [Simulator-byteCountHelp.1.cs](#): Miscellaneous minor fixes and documentation improvements.
- **OpenSmalltalk VM**
 - [OpenSmalltalk/opensmalltalk-vm#716](#): Revise installation instructions for Ubuntu
 - [OpenSmalltalk/opensmalltalk-vm#719](#): Add support for clang>=16 [v2]
 - [OpenSmalltalk/opensmalltalk-vm#725](#): Add build files for linux64x64/squeak.sista.spur
 - [OpenSmalltalk/opensmalltalk-vm#728](#): [ci] add squeak.sista.spur build for linux64x64

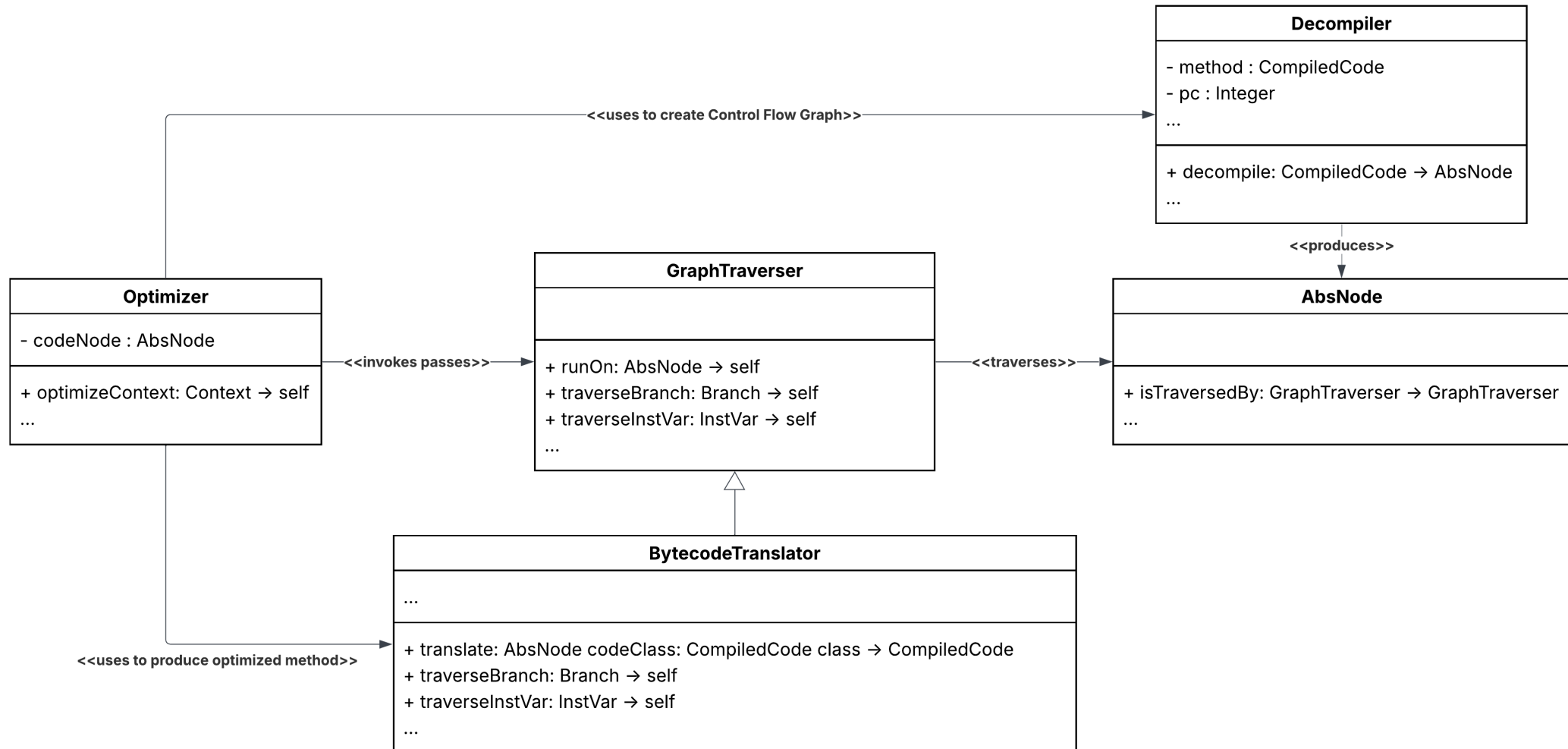
OSVM: VMMaker Architecture

Simplified significantly, overview of most important classes only



OSVM: Scorch Architecture

Simplified significantly, overview of most important classes for optimization only



TSVM: Data Representation

How are Squeak objects represented in Java?

Primitive instances

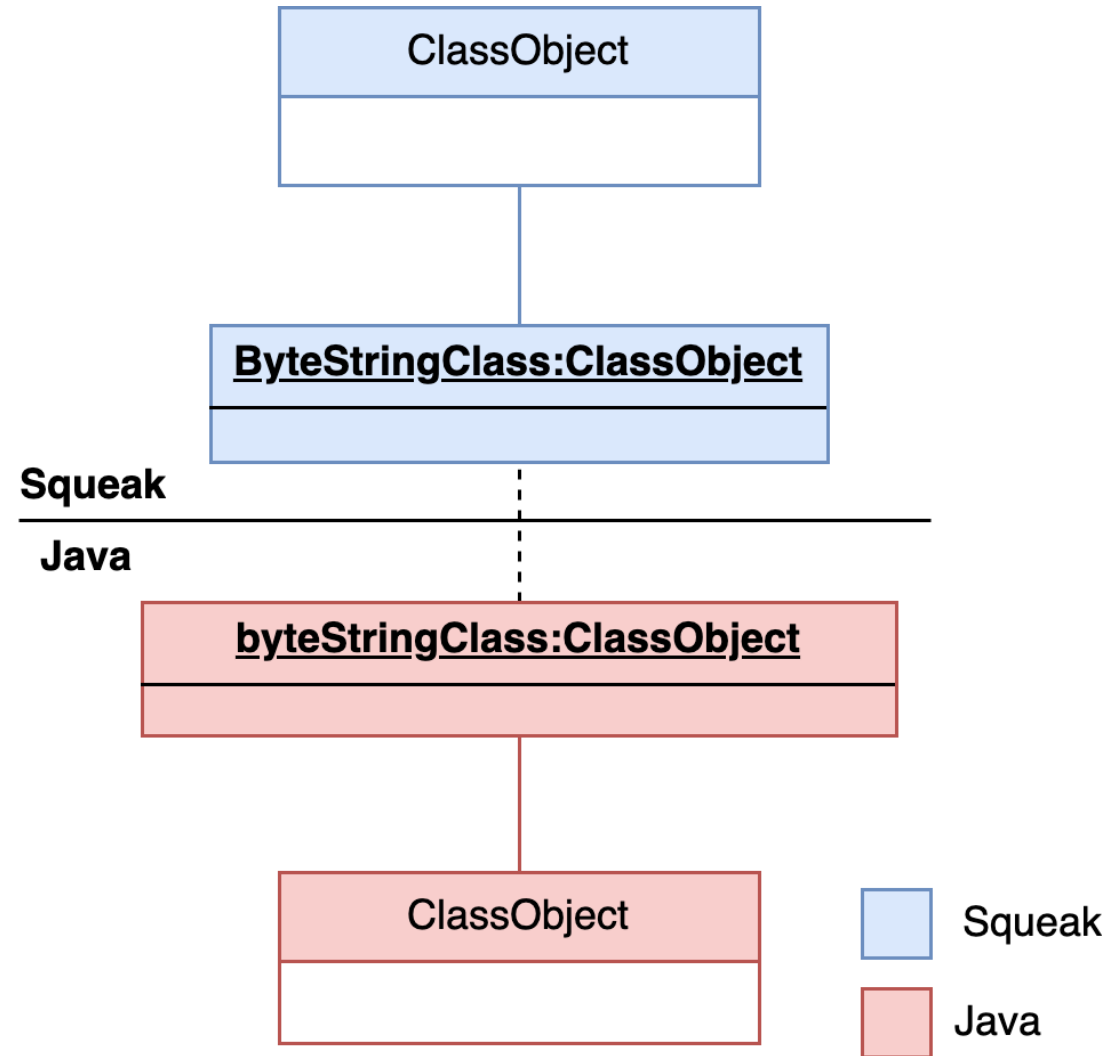


Image: UML Class Diagram of Primitive Representation

TSVM: Architecture

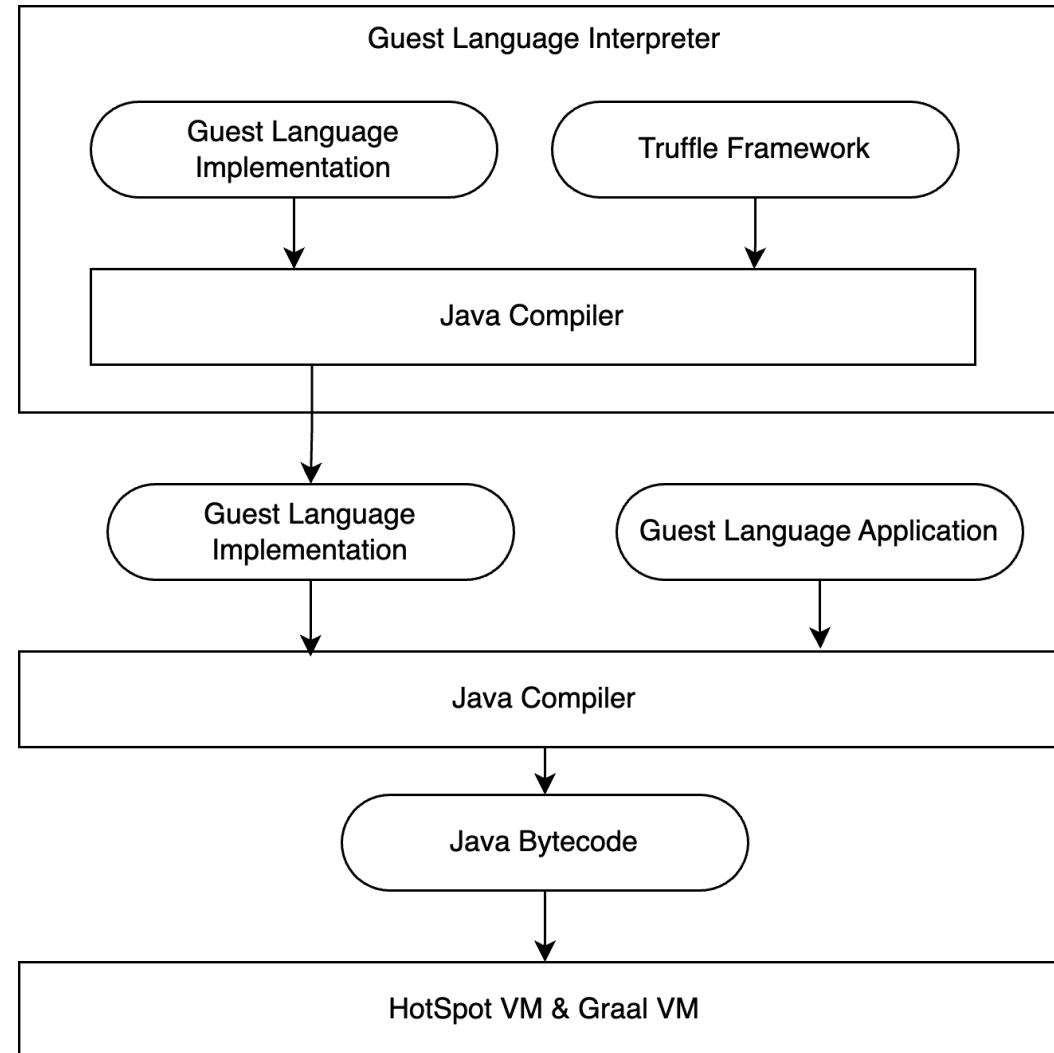


Image: FMC Diagram of used Compiler

TSVM: Architecture

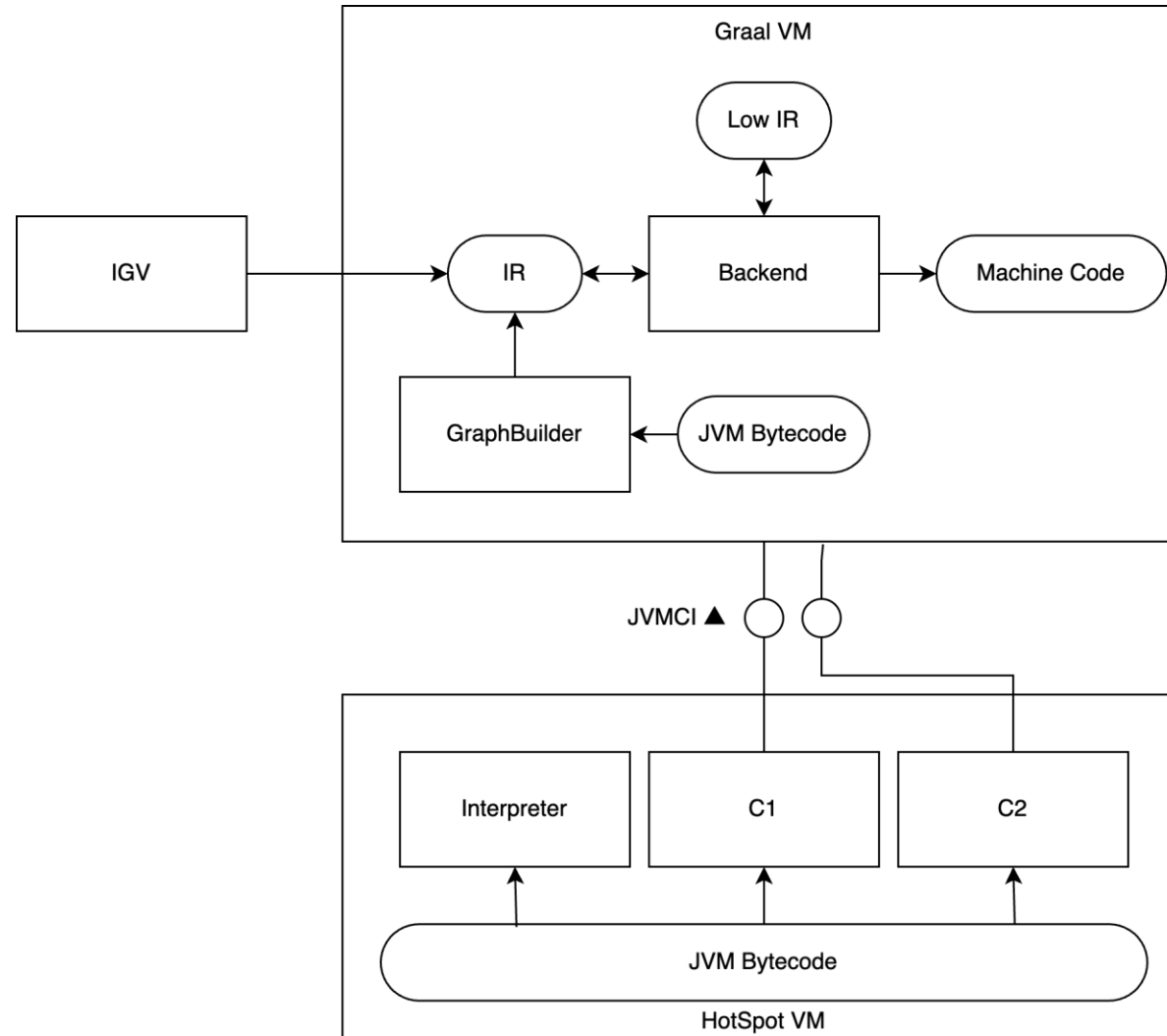


Image: FMC Diagram of GraalVM

TSVM: ByteArray as TruffleString

v2

HPI

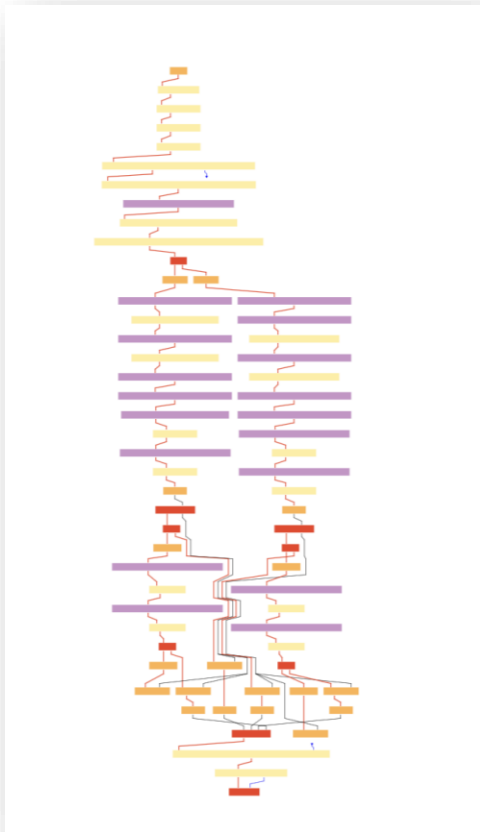


Image: Compilation Graph Original

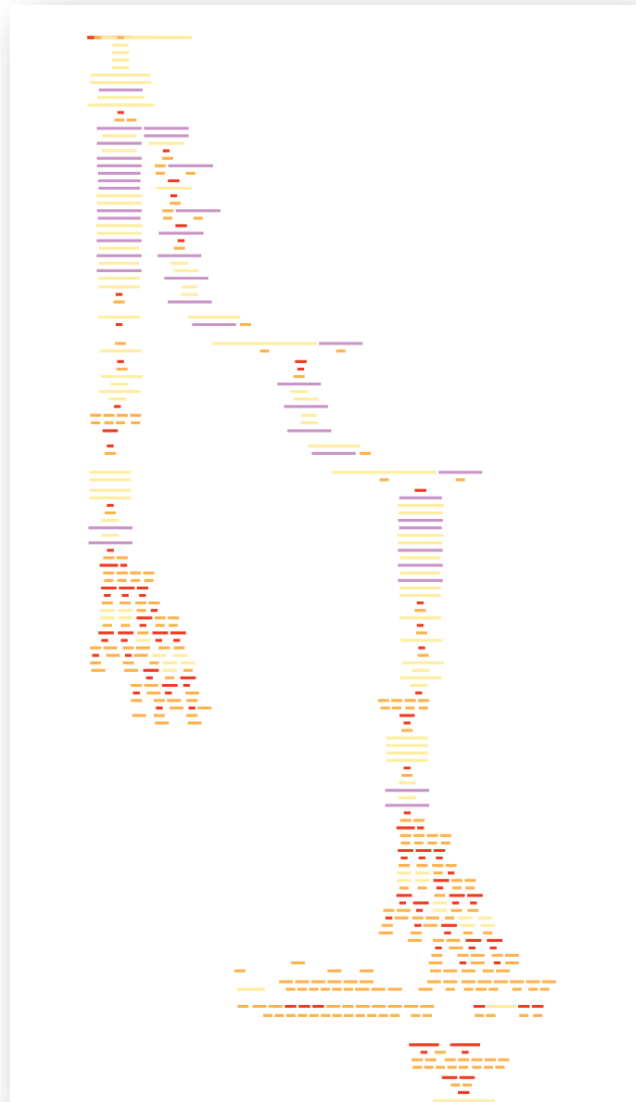


Image: Compilation Graph v1

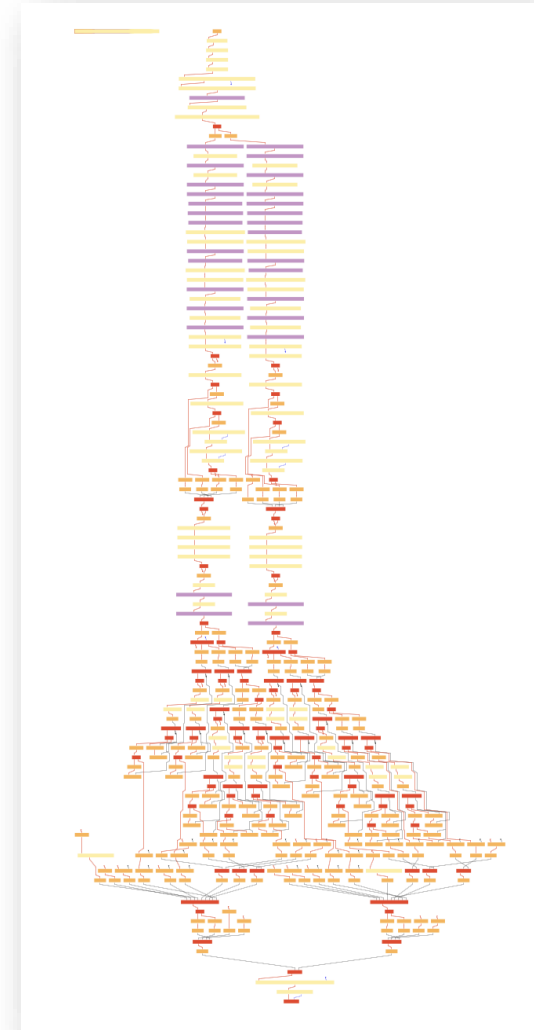


Image: Compilation Graph: v2